

Tu código describe un divisor algorítmico en Verilog que implementa la división secuencial con registro acumulador (ACCU). Este enfoque es típico en hardware para realizar divisiones de números grandes o cuando se desea evitar el uso de bloques divisores predefinidos. A continuación, haré algunos comentarios sobre tu código para mejorar su funcionalidad y corregir errores potenciales.

1. Estado Finito y Transiciones

En tu máquina de estados, estás usando la enumeración `enum {d0, d1, d2, d3} state;`, pero parece que falta declarar el estado siguiente (`next_state`), que normalmente ayuda a mantener las transiciones claras. Sin embargo, dado que en tu código actual las transiciones de estado están dentro de un `always_ff`, puedes mantenerlo sin un `next_state` explícito si prefieres esa estructura.

Aun así, sugiero agregar una declaración explícita para el estado `state` y reinicializarlo en `d0` durante el reset:

verilog

Copiar código

```
logic [1:0] state; // Se almacena el estado actual
```

Luego, en el bloque `always_ff`, puedes inicializar `state <= d0`; dentro del bloque de reset.

2. Asignación de fin

El uso de la señal `fin` como indicador de "Done" es correcto, pero hay un pequeño detalle. En el estado `d2`, no parece estar claro cuándo `fin` se pondrá en alto para indicar que el cálculo ha terminado. El hecho de que `fin` solo se ponga en alto en el estado `d0` puede causar comportamientos inconsistentes si el valor de `Done` se requiere justo después de que se complete el cálculo.

Sería útil mover la asignación de `Done` al final del ciclo para que se active cuando realmente haya terminado el cálculo:

verilog

Copiar código

```
assign Done = (state == d3);
```

3. Corrección de Count

En el estado `d1`, hay una operación `Count <= Count + 1`, pero esto debería ser `Count <= Count - 1`; ya que estás haciendo una cuenta regresiva desde `tamanyo-1` hasta 0. Además, es importante asegurarte de que el ancho del contador esté correctamente definido, y en tu caso, puede ser problemático.

Usa:

verilog

Copiar código

```
logic [$clog2(tamanyo):0] Count;
```

para asegurar que el contador tenga suficiente ancho para manejar todos los ciclos.

4. Desplazamiento del ACCU y Q

En el estado d1, tienes esta línea:

verilog

Copiar código

```
{ACCU, Q} <= {ACCU[tamanyo-1], Q, 1'b0};
```

Esto puede resultar en una incorrecta concatenación y desplazamiento. Deberías asegurarte de que estás desplazando correctamente ambos registros, y podrías hacer algo más explícito como:

verilog

Copiar código

```
{ACCU, Q[tamanyo-1:1]} <= {ACCU[tamanyo-2:0], Q[tamanyo-1], 1'b0};
```

Aquí estamos desplazando ACCU a la izquierda y moviendo el bit más significativo de Q al bit menos significativo de ACCU, además de desplazar Q un bit a la izquierda.

5. Manejo de signos

La lógica para manejar el signo está correctamente encaminada, pero el manejo del número negativo se realiza en varios lugares. En d3, se ajusta el cociente y el residuo dependiendo del signo de los operandos de entrada. Esto es adecuado, pero asegúrate de que el ajuste se haga de forma consistente para evitar errores de implementación.

Por ejemplo, cuando se ajusta el cociente y el residuo:

verilog

Copiar código

```
Coc <= (SignNum ^ SignDen) ? (~Q + 1) : Q;
```

```
Res <= SignNum ? (~ACCU + 1) : ACCU;
```

Esto debería funcionar bien siempre que SignNum y SignDen se configuren correctamente en d0.

6. Valor de Inicialización de Señales

Asegúrate de que todas las señales estén correctamente inicializadas, especialmente en el bloque de reset. Esto incluye ACCU, Q, M, y los signos. Además, en el bloque de default, reinicializa adecuadamente las señales.

Código Mejorado

Aquí te dejo una versión mejorada con los puntos anteriores:

verilog

Copiar código

```
module Divisor_Algoritmico
#(parameter tamanyo=32)
(
    input CLK,
    input RSTa,
    input Start,
    input logic [tamanyo-1:0] Num,
    input logic [tamanyo-1:0] Den,
    output logic [tamanyo-1:0] Coc,
    output logic [tamanyo-1:0] Res,
    output Done
);

    logic SignNum, SignDen, fin;
    logic [tamanyo-1:0] ACCU, Q, M;
    logic [$clog2(tamanyo):0] Count;
    logic [1:0] state;

    assign Done = (state == d3);
```

```

always_ff @(posedge CLK or negedge RSTa) begin
    if (!RSTa) begin
        state <= d0;
        {ACCU, Q, M, SignNum, SignDen, Count} <= 0;
    end else begin
        case (state)
            d0: begin
                fin <= 0;
                if (Start) begin
                    ACCU <= 0;
                    Count <= tamanyo - 1;
                    SignNum <= Num[tamanyo-1];
                    SignDen <= Den[tamanyo-1];
                    Q <= Num[tamanyo-1] ? (~Num + 1) : Num;
                    M <= Den[tamanyo-1] ? (~Den + 1) : Den;
                    state <= d1;
                end else begin
                    state <= d0;
                end
            end
            d1: begin
                fin <= 0;
                {ACCU, Q[tamanyo-1:1]} <= {ACCU[tamanyo-2:0], Q[tamanyo-1], 1'b0};
                state <= d2;
            end
        endcase
    end
end

```

```

d2: begin
    fin <= 0;
    if (ACCU >= M) begin
        Q <= Q + 1;
        ACCU <= ACCU - M;
    end
    Count <= Count - 1;
    if (Count == 0) state <= d3;
    else state <= d1;
end

d3: begin
    Coc <= (SignNum ^ SignDen) ? (~Q + 1) : Q;
    Res <= SignNum ? (~ACCU + 1) : ACCU;
    state <= d0;
end

default: begin
    state <= d0;
    {ACCU, Q, M, SignNum, SignDen, Count} <= 0;
end
endcase
end
end

endmodule

```

Con estos ajustes, el módulo debería funcionar de manera más precisa y eficiente.

Tú dijiste:

me puedes explicar que hace cada uno de los estados d0 d1 d2 d3

ChatGPT Plus



¡Claro! Cada uno de los estados de tu máquina de estados en el divisor algorítmico tiene un papel clave en la ejecución del proceso de división secuencial. Vamos a repasar en detalle lo que ocurre en cada uno de ellos:

Estado d0: Inicialización y Preparación

Este estado se encarga de inicializar todas las variables y realizar las configuraciones necesarias antes de comenzar la división.

1. Verificación de la señal Start:

- Si la señal de inicio (Start) es alta, el divisor comienza su operación. Si no, el estado permanece en d0 esperando la señal de inicio.

2. Inicialización de registros:

- El acumulador (ACCU) se inicializa en 0.
- El contador (Count) se inicializa en tamaño-1 (es decir, el número de bits menos uno).

3. Manejo de signos:

- SignNum y SignDen almacenan los signos del numerador (Num) y del denominador (Den), respectivamente, extrayendo el bit más significativo (MSB) de cada uno, que es el bit de signo en la representación en complemento a dos.

4. Preparación de los valores absolutos:

- Q toma el valor absoluto del numerador: Si el número es negativo, se obtiene el complemento a dos de Num ($\sim\text{Num} + 1$).
- M toma el valor absoluto del denominador: Si el denominador es negativo, también se obtiene su complemento a dos ($\sim\text{Den} + 1$).

5. Transición al siguiente estado:

- Si Start es alta, la máquina de estados pasa a d1.

Este estado asegura que todos los valores necesarios están listos para iniciar el proceso de división.

Estado d1: Desplazamiento a la Izquierda y Expansión

Este es el primer paso en el proceso de división propiamente dicho, donde se realiza el desplazamiento de los valores para preparar la resta de los bits más significativos.

1. Desplazamiento a la izquierda:

- Se realiza un **desplazamiento a la izquierda** conjunto del acumulador (ACCU) y el numerador (Q).
- El bit más significativo del acumulador (ACCU[tamaño-1]) se mueve al bit menos significativo de Q. El bit menos significativo de Q recibe un 0 adicional para mantener el formato.

El objetivo de este desplazamiento es preparar el acumulador para realizar la resta en el siguiente ciclo.

2. Transición al siguiente estado:

- Después del desplazamiento, el estado cambia a d2.

Estado d2: Comparación y Resta

En este estado ocurre el corazón de la operación de división, donde se realiza la resta condicional entre el acumulador (ACCU) y el denominador (M).

1. Comparación:

- Si el acumulador (ACCU) es mayor o igual que el denominador (M), significa que la división puede realizarse para el bit más significativo.

2. Resta condicional:

- Si la condición anterior es verdadera (es decir, $ACCU \geq M$), se realiza la resta $ACCU \leftarrow ACCU - M$ y se actualiza el cociente parcial sumando 1 al bit menos significativo de Q ($Q \leftarrow Q + 1$).

3. Decremento del contador:

- El contador (Count) se decrementa en cada ciclo, ya que se está procesando un bit de la división en cada iteración.

4. Verificación de finalización:

- Si el contador (Count) ha llegado a 0, significa que se ha completado el proceso de división para todos los bits, y la máquina de estados transiciona a d3 para concluir el cálculo.

- Si el contador no ha llegado a 0, se vuelve al estado d1 para continuar con la siguiente iteración del proceso.

Estado d3: Ajuste de Signos y Finalización

Este estado se encarga de ajustar los signos del cociente y el residuo, y marcar la finalización de la operación.

1. Ajuste del cociente:

- Si los signos del numerador y del denominador son diferentes (es decir, $\text{SignNum} \wedge \text{SignDen}$ es verdadero), se invierte el cociente Q (cociente parcial calculado) utilizando complemento a dos: $\text{Coc} \leftarrow \sim Q + 1$. Esto se hace para reflejar el signo correcto del resultado.
- Si los signos son iguales, se asigna el valor de Q directamente a Coc.

2. Ajuste del residuo:

- Si el numerador es negativo (SignNum es verdadero), se toma el complemento a dos del residuo (ACCU): $\text{Res} \leftarrow \sim \text{ACCU} + 1$. De lo contrario, se asigna ACCU directamente a Res.

3. Finalización:

- El estado transiciona de nuevo a d0, y la señal Done se activa indicando que la operación ha concluido.

Resumen del Flujo de Estados:

1. d0 (Inicialización y preparación):

- Inicia la operación, ajusta signos y convierte los operandos a sus valores absolutos.

2. d1 (Desplazamiento y expansión):

- Desplaza el acumulador y el cociente para preparar la comparación y resta.

3. d2 (Comparación y resta):

- Realiza la resta condicional y actualiza el cociente, decrece el contador para cada bit.

4. d3 (Ajuste de signos y finalización):

- Ajusta los signos del cociente y el residuo, marca el final del proceso.

Este flujo es típico en un divisor secuencial. Cada ciclo de reloj procesa un bit de la división, lo que lo convierte en un divisor eficiente en cuanto a área pero más lento en comparación con divisores combinacionales.

Tú dijiste:

puedes hacerme un test bench simple para ver como funciona

ChatGPT Plus



¡Claro! A continuación te proporciono un testbench básico para tu módulo de división algorítmica en Verilog. Este testbench simula diferentes casos de división, verificando tanto el cociente como el residuo para números positivos y negativos.

Testbench para el Módulo Divisor_Algoritmico

verilog

Copiar código

```
module tb_Divisor_Algoritmico;

    // Parámetro de tamaño de los datos
    parameter tamanyo = 32;

    // Señales de entrada
    reg CLK;
    reg RSTa;
    reg Start;
    reg [tamanyo-1:0] Num; // Numerador
    reg [tamanyo-1:0] Den; // Denominador

    // Señales de salida
    wire [tamanyo-1:0] Coc; // Cociente
    wire [tamanyo-1:0] Res; // Residuo
```

```
wire Done;          // Señal de finalización
```

```
// Instancia del módulo bajo prueba (Divisor_Algoritmico)
```

```
Divisor_Algoritmico #(tamanyo) uut (
```

```
    .CLK(CLK),
```

```
    .RSTa(RSTa),
```

```
    .Start(Start),
```

```
    .Num(Num),
```

```
    .Den(Den),
```

```
    .Coc(Coc),
```

```
    .Res(Res),
```

```
    .Done(Done)
```

```
);
```

```
// Generación del reloj
```

```
always #5 CLK = ~CLK; // Periodo de 10 unidades de tiempo (5 unidades high, 5  
low)
```

```
// Procedimiento de inicialización y test
```

```
initial begin
```

```
    // Inicialización de señales
```

```
    CLK = 0;
```

```
    RSTa = 0;
```

```
    Start = 0;
```

```
    Num = 0;
```

```
    Den = 0;
```

```
// Reset del sistema
```

```
#10 RSTa = 1; // Activar reset asíncrono
```

```
#10 RSTa = 0; // Quitar reset
```

```
#10 RSTa = 1; // Vuelve al estado normal
```

```
// Caso 1: División positiva 100 / 3
```

```
#10 Num = 100; Den = 3; Start = 1;
```

```
#10 Start = 0; // Retirar la señal de inicio después de un ciclo
```

```
wait(Done); // Espera a que la señal de "Done" se active
```

```
#10; // Espera un ciclo después de que "Done" esté en alto
```

```
$display("División 100 / 3: Coc = %d, Res = %d", Coc, Res);
```

```
// Caso 2: División negativa -100 / 3
```

```
#10 Num = -100; Den = 3; Start = 1;
```

```
#10 Start = 0;
```

```
wait(Done);
```

```
#10;
```

```
$display("División -100 / 3: Coc = %d, Res = %d", Coc, Res);
```

```
// Caso 3: División positiva 45 / -5
```

```
#10 Num = 45; Den = -5; Start = 1;
```

```
#10 Start = 0;
```

```
wait(Done);
```

```
#10;
```

```
$display("División 45 / -5: Coc = %d, Res = %d", Coc, Res);
```

```
// Caso 4: División negativa -45 / -5
```

```
#10 Num = -45; Den = -5; Start = 1;
```

```
#10 Start = 0;
```

```

wait(Done);

#10;

$display("División -45 / -5: Coc = %d, Res = %d", Coc, Res);


// Caso 5: División con residuo 37 / 4

#10 Num = 37; Den = 4; Start = 1;

#10 Start = 0;

wait(Done);

#10;

$display("División 37 / 4: Coc = %d, Res = %d", Coc, Res);


// Fin de la simulación

#50;

$finish;

end

endmodule

```

Explicación del Testbench

1. Señales de Entrada y Salida:

- El testbench define las señales que se conectan al divisor: CLK (reloj), RSTa (reset asíncrono), Start (señal de inicio), Num (numerador), y Den (denominador).
- Las señales de salida son Coc (cociente), Res (residuo) y Done (señal que indica que la operación ha terminado).

2. Generación del Reloj:

- El reloj (CLK) se genera usando un always que invierte su valor cada 5 unidades de tiempo. Esto crea un ciclo de reloj de 10 unidades de tiempo (5 unidades en alto y 5 en bajo).

3. Secuencia de Prueba:

- Primero se aplica un reset durante 10 unidades de tiempo (RSTa = 0), luego se libera (RSTa = 1).

- Después, se aplican diferentes valores de entrada a Num y Den para probar varias divisiones, y la señal Start se activa durante un ciclo de reloj para iniciar el cálculo.
- Después de cada división, se espera que la señal Done se active antes de mostrar los resultados.

4. Casos de Prueba:

- **Caso 1:** División de 100 / 3.
- **Caso 2:** División de -100 / 3 (numerador negativo).
- **Caso 3:** División de 45 / -5 (denominador negativo).
- **Caso 4:** División de -45 / -5 (numerador y denominador negativos).
- **Caso 5:** División de 37 / 4 para verificar un residuo no nulo.

5. Visualización de Resultados:

- Después de que se complete cada división (cuando Done se pone en alto), el testbench muestra los valores de cociente y residuo con un display.

Cómo usar el testbench:

1. **Simulación:** Compila y ejecuta este testbench en cualquier simulador Verilog, como ModelSim, VCS o cualquier otro simulador de Verilog que prefieras.
2. **Revisar la salida:** Verás en la consola los valores del cociente y residuo para cada caso de prueba. Deberías obtener resultados similares a los siguientes (dependiendo del simulador):

text

Copiar código

División 100 / 3: Coc = 33, Res = 1

División -100 / 3: Coc = -33, Res = -1

División 45 / -5: Coc = -9, Res = 0

División -45 / -5: Coc = 9, Res = 0

División 37 / 4: Coc = 9, Res = 1

Nota:

Este testbench es simple y está diseñado para verificar si el divisor funciona correctamente con diferentes valores de entrada. Puedes agregar más casos de prueba, o mejorar el testbench para incluir más automatización, por ejemplo, usando bucles para probar múltiples valores automáticamente.

ChatGPT puede